ESTRUCTURA DE DATOS · ESPE

Complejidad Algorítmica

Análisis experimental en el proyecto Royal Casino

Midiendo con performance.now() si el tiempo de ejecución es constante u O(n)



Control de acceso



Ruleta



Blackjack

¿Qué vamos a demostrar?

El proyecto Royal Casino tiene funciones que se ejecutan miles de veces por segundo. Para saber qué tan bien escalan, no basta con mirar el código: hay que cronometrarlas. Usamos performance.now() en tres funciones reales del casino y comparamos el tiempo medido contra lo que predice la teoría.

O(1)

Tiempo constante

Por más que crezca n, cada llamada hace siempre el mismo número de pasos. El tiempo por operación no se mueve.

O(n)

Tiempo lineal

El trabajo crece junto con n: el doble de datos significa, más o menos, el doble de tiempo.

¿Cómo medimos?

1

Marca de inicio

`t0 = performance.now()`

→

2

Se ejecuta la función

`con un tamaño de entrada n`

→

3

Marca de fin y resta

`tiempo total = t1 − t0`

→

4

Se promedia

`tiempo total ÷ n = tiempo por operación`

Fórmulas confirmadas en cada función

Función	Fórmula T(n)	Clase
Control de acceso	`T(n) = 1`	**O(1)**
Ruleta — dibujo de la rueda	`T(n) = n  (n = casillas)`	**O(n)**
Blackjack — cálculo de puntaje	`T(n) = n  (n = cartas)`	**O(n)**

CASO 1 · O(1)

Control de acceso

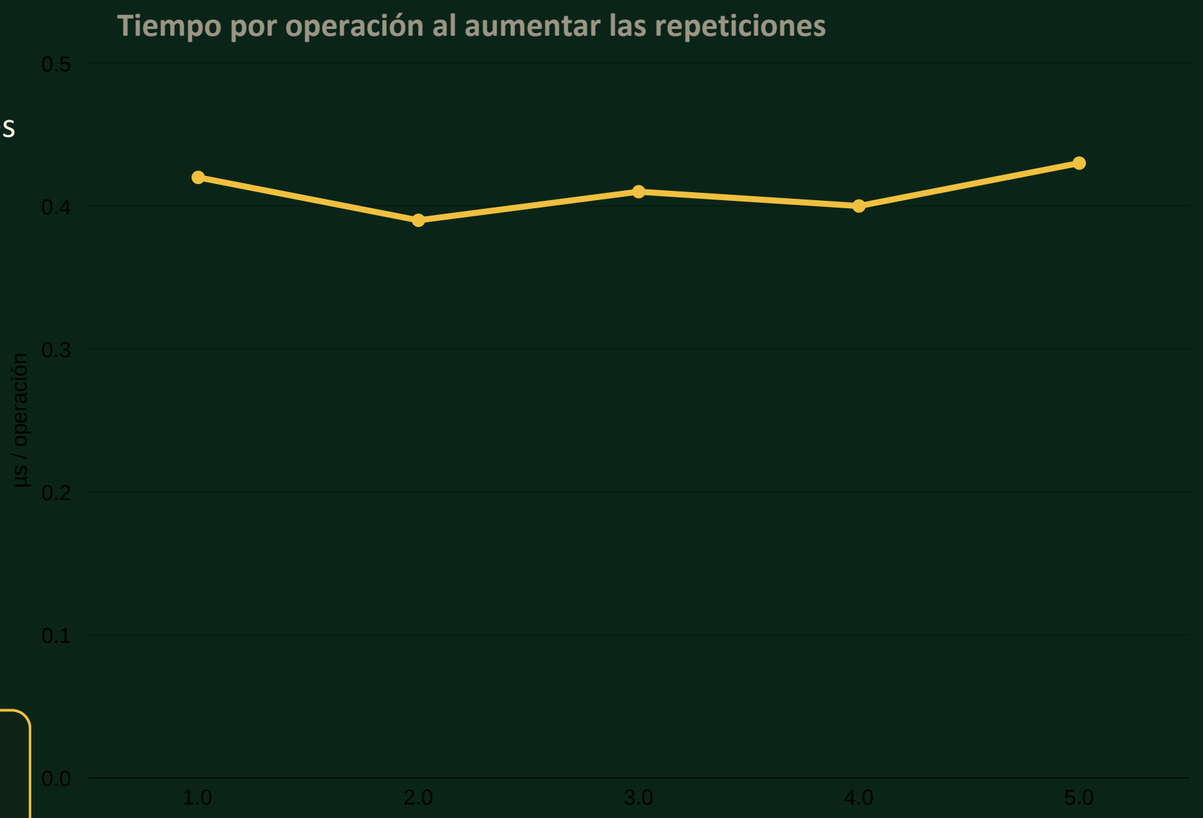
¿QUÉ HACE?

Revisa si un jugador puede entrar al casino verificando 4 condiciones fijas: edad, veto, identificación y reglas de mesa.

¿POR QUÉ ESE COMPORTAMIENTO?

Sin importar cuántas veces se llame, siempre evalúa las mismas 4 condiciones. No hay ningún bucle que dependa del tamaño de los datos: el trabajo no crece.

$$T(n) = 1$$



Valores ilustrativos. La línea se mantiene plana: más repeticiones, mismo costo por operación.

CASO 2 · O(N)

Ruleta — dibujo de la rueda

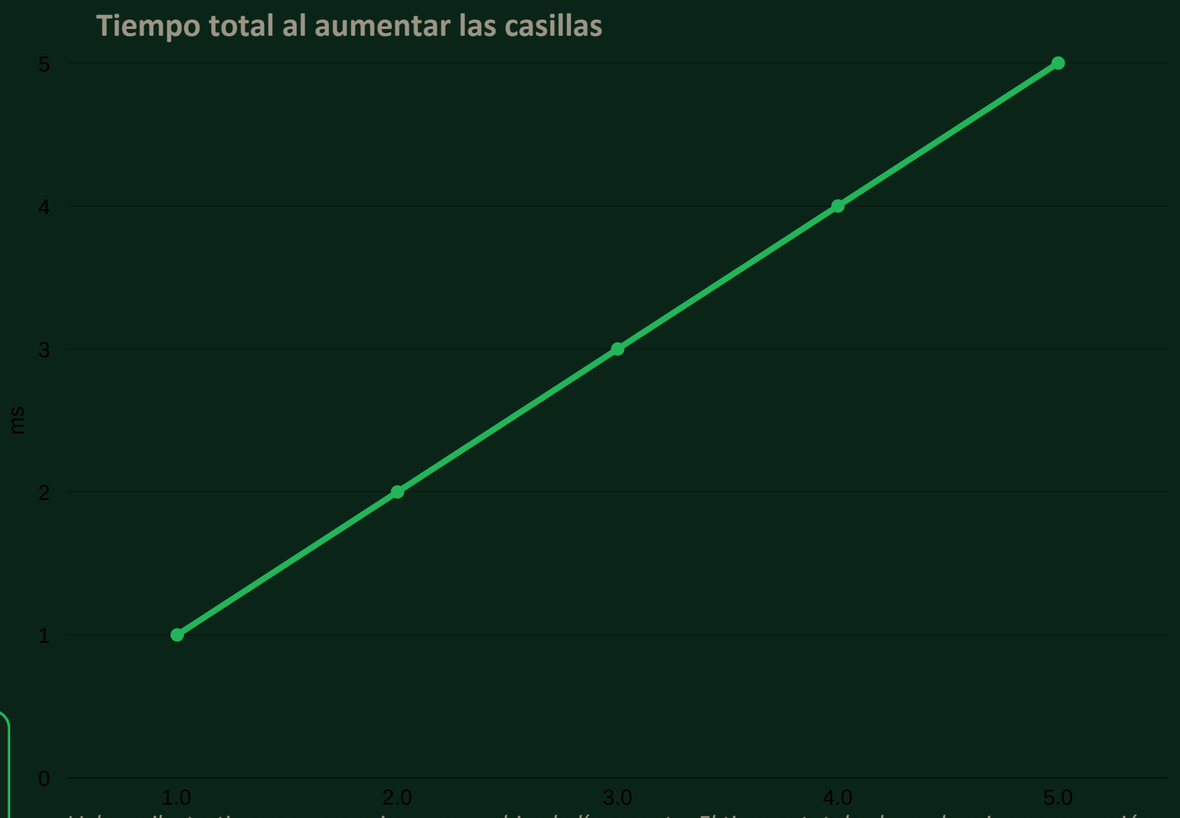
¿QUÉ HACE?

Dibuja la ruleta casilla por casilla, calculando posición y color para cada uno de los números del 0 al 36 (o más, en la prueba).

¿POR QUÉ ESE COMPORTAMIENTO?

Hay un bucle que repite el mismo dibujo una vez por casilla. Si la rueda tuviera 1.000 casillas en vez de 37, el bucle se repite 1.000 veces: a más casillas, más tiempo, en la misma proporción.

```
T(n) = n    (casillas)
```



Valores ilustrativos con n parejo, para ver bien la línea recta. El tiempo total sube en la misma proporción que n.

CASO 3 · O(N)

Blackjack — cálculo de puntaje

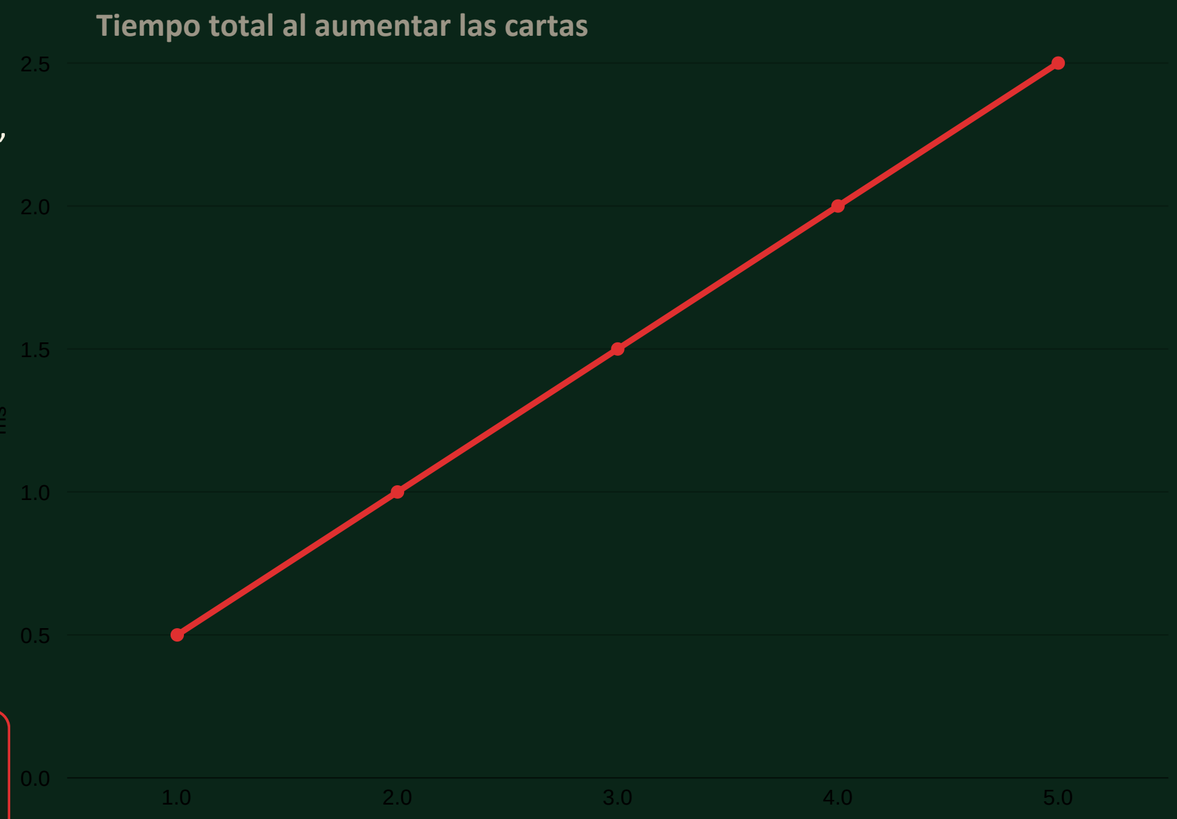
¿QUÉ HACE?

Suma el valor de cada carta en la mano del jugador (As=11, J/Q/K=10, el resto su número) y ajusta los ases si el total se pasa de 21.

¿POR QUÉ ESE COMPORTAMIENTO?

Recorre carta por carta para sumar: con 2 cartas suma 2 veces, con 10.000 cartas suma 10.000 veces. El trabajo crece exactamente al ritmo de las cartas.

$$T(n) = n \quad \text{(cartas)}$$



Valores ilustrativos con n parejo, para ver bien la línea recta. Misma tendencia que la ruleta, a otra escala.

Una trampa común

Repetir cualquier operación más veces siempre tarda más en total. Eso no prueba nada por sí solo: hay que distinguir entre repetir una llamada y aumentar el tamaño real de la entrada.

Repetir n veces

Control de acceso

- El tiempo TOTAL sube porque son más llamadas.
- El tiempo POR operación se mantiene igual.
- **Conclusión correcta: sigue siendo O(1).**

Crecer la entrada n

Ruleta y Blackjack

- n es el tamaño real de los datos que procesa la función.
- El tiempo total crece en la misma proporción que n.
- **Conclusión correcta: es O(n).**

CONCLUSIÓN

Tres funciones, dos comportamientos

Función	T(n)	Complejidad confirmada
Control de acceso	`1`	**O(1) — constante**
Ruleta (dibujo)	`n`	**O(n) — lineal**
Blackjack (puntaje)	`n`	**O(n) — lineal**

La teoría predice cómo debería comportarse una función. Medir con performance.now() nos deja comprobarlo con números reales, no solo con la fórmula en el papel.